

Java Advanced Schulung 2026

Lea Föcke & Phil Steinhorst



Lambda-Ausdrücke

- Lambda-Ausdrücke sind anonyme (= namenlose, ungebundene) Funktionen.
- **Funktion:** Oberbegriff für Methode, aber nicht notwendig zu einer Klasse gehörend.
- Allgemeine Syntax: `(param1, param2, ..., paramN) -> {Code-Block}`
- Bei nur einem Parameter können die Klammern fortgelassen werden:
`param -> {Code-Block}`
- Ebenso bei nur einer Anweisung im Code-Block (analog `if`, `while`, etc.)
`(param1, param2, ... paramN) -> Anweisung`

Verwendung von Lambda-Ausdrücken

- Häufige Verwendung für "kurze" Berechnungsvorschriften.

Als Methode:

```
public double mittelwert (double x, double y) {  
    return (x + y) / 2;  
}
```

Als Lambda-Ausdruck:

```
(x, y) -> (x + y) / 2;
```


- Lambda-Ausdrücke können – wie andere Werte auch – als Parameter und Rückgabe verwendet sowie Bezeichnern zugewiesen werden:

```
List<Integer> ls = List.of(4, 8, 15);  
ls.forEach( x -> System.out.println(x * 2) );  
// Ausgabe:  
8  
16  
30
```

- Welchen Datentyp hat der Lambda-Ausdruck?

Datentypen von Lambda-Ausdrücken

forEach

```
public void forEach(Consumer<? super E> action)
```

- Die Methode `forEach` von `List<E>` verlangt ein `Consumer<? super E>`.

```
@FunctionalInterface
```

```
public interface Consumer<T>
```

Represents an operation that accepts a single input argument and returns no result. Unlike most other functional interfaces, `Consumer` is expected to operate via side-effects.

- `Consumer<T>` ist ein **functional interface** für Lambda-Ausdrücke mit einem Parameter von Typ `T` und ohne Ergebnis (`void`).

Speichern von Lambda-Ausdrücken

- Wir können `Consumer` nutzen, um einen Lambda-Ausdruck zu speichern...

```
Consumer<Integer> fun = x -> System.out.println(2 * x);  
List<Integer> ls = List.of(4, 8, 15);  
ls.forEach(fun);
```

- ... oder um ihn als Resultat einer Methode zurückzugeben:

```
public Consumer<Integer> nMalAusgeben(int n) {  
    return x -> {  
        for (int i = 0; i < n; i++)  
            System.out.println(x);  
    };  
}
```

```
public Consumer<Integer> nMalAusgeben(int n) {  
    return x -> {  
        for (int i = 0; i < n; i++)  
            System.out.println(x);  
    };  
}
```

```
var dreiMalAusgeben = nMalAusgeben(3);  
dreiMalAusgeben.accept(5);  
// Ausgabe:  
5  
5  
5
```

Functional Interfaces

- `@FunctionalInterface` zeigt an, dass das Interface **genau** eine abstrakte Methode (= nicht `@Override` und ohne `default`-Implementierung) definiert.
- Diese legt fest, welche Lambda-Ausdrücke verwendet werden können

```
@FunctionalInterface
public interface Consumer<T> {
    // abstrakt, da ohne default-Implementierung
    // Eingabe: Objekt vom Typ T; Rückgabe: nichts (void)
    void accept(T t);

    // nicht abstrakt, da default-Implementierung vorhanden
    default Consumer<T> andThen(Consumer<? super T> after) {
        /* ... */
    }
}
```


- Definiert man in einem functional interface mehr als eine (oder gar keine) abstrakte Methode, führt dies zu einem Compile-Fehler.

```
@FunctionalInterface
public interface NichtFunktional {
    int eineMethode(int a, int b);
    double andereMethode(double a, double b);
}
```

```
java: Unexpected @FunctionalInterface annotation
NichtFunktional is not a functional interface
multiple non-overriding abstract methods found in interface NichtFunktional
```

- Ganz allgemein kann man **jedes** Interface, das genau eine abstrakte Methode definiert, durch einen Lambda-Ausdruck instanziiieren:

```
// mit oder ohne @FunctionalInterface-Annotation
public interface MeinInterface{
    int machWas(int x);
}

MeinInterface fun = x -> 2*x;
fun.machWas(5); // Rückgabe: 10
```

- Die Annotation verhindert nur, dass man "versehentlich" eine weitere abstrakte Methode definiert...
- ... und weist Entwickler*innen darauf hin, dass man bequem einen Lambda-Ausdruck verwenden kann.

Was passiert beim Instanziieren?

```
public interface MeinInterface{  
    int machWas(int x);  
}  
  
MeinInterface fun = x -> 2*x;
```

- Technisch wird zur Laufzeit eine anonyme Klasse erzeugt, die das Interface implementiert und dafür den Rumpf des Lambda-Ausdrucks verwendet.

```
class OhneName implements MeinInterface{  
    int machWas(int x) {  
        return 2*x;  
    }  
}  
  
OhneName fun = new OhneName();
```

- In `java.util.function` sind Functional Interfaces für übliche Kombinationen von Eingaben und Rückgaben vordefiniert:
 - `Function<T,R>` : ein Parameter vom Typ `T` ; Rückgabe vom Typ `R` .
 - `BiFunction<T,U,R>` : ein Parameter vom Typ `T` , einer vom Typ `U` ; Rückgabe vom Typ `R` .
 - `BinaryOperator<T>` : zwei Parameter und Rückgabe vom Typ `T` .
 - ... (siehe [Doku zu `java.util.function`](#))
- Da Java nicht erraten kann, welches Interface man implementieren möchte: Lambda-Ausdrücke müssen immer **explizit** typisiert werden.

```
var mittel = (a, b) -> (a+b) / 2;
```

Cannot infer type: lambda expression requires an explicit target type

Nicht vordefinierte Functional Interfaces

- Für darüber hinausgehende Lambda-Ausdrücke muss man selbst ein Functional Interface definieren.

```
/**
 * Funktionales Interface für Lambda-Ausdrücke mit drei Parametern
 * und Rückgabe vom selben Typ.
 */
@FunctionalInterface
public interface TernaryOperator<T> {
    T apply(T a, T b, T c);
}

TernaryOperator<Double> mittel = (a, b, c) -> (a + b + c) / 3;
mittel.apply(5.0, 13.0, 14.0);
```

Methodenreferenzen

- Methoden können referenziert und genauso verwendet werden wie Lambda-Ausdrücke.
- Syntax für nicht-statische Methoden: `objekt::methode`

```
public class Foo {  
    public void doppelteAusgeben(int x) {  
        System.out.println(2 * x);  
    }  
}
```

```
List<Integer> ls = List.of(4, 8, 15);  
Foo f = new Foo();  
ls.forEach( f::doppelteAusgeben );
```

- Syntax für statische Methoden: `Klasse::methode`

Mehr Infos zu Lambda-Ausdrücken

- <https://www.baeldung.com/java-8-functional-interfaces>
- <https://www.baeldung.com/java-8-lambda-expressions-tips>

Offene Fragen?

Streams

- **Streams** (dt. *Datenströme*) sind Datenstrukturen, die sich für eine **deklarative Verarbeitung** von Elementen eignen.
- *Deklarativ* bedeutet: Statt (wie z.B. in Methoden) die Berechnung anzugeben, spezifiziert man das Ergebnis (**was** statt **wie**).
- Dabei helfen vor allem Lambda-Ausdrücke.
- Die Verarbeitung ist dadurch in vielen Fällen besonders gut lesbar und nachvollziehbar.
- Streams eignen sich von Haus aus für eine parallele Verarbeitung.

Erzeugung von Streams

- Erzeugung aus einer `Collection`:

```
Stream<Integer> s = List.of(4, 8, 15, 16, 23, 42).stream();
```

- Mittels `Stream.of(...)`:

```
Stream<Integer> s = Stream.of(4, 8, 15, 16, 23, 42);
```

- sieht man zum Beispiel bei parametrisierten JUnit-Tests

- Mittels *Builder Pattern*:

```
Stream<Integer> s = Stream.<Integer>builder().add(4).add(8).add(15).build();
```

- Mittels Generator: `Stream.generate(Supplier<T> fun)`
 - `fun` : Lambda-Ausdruck (oder Methode) ohne Parameter.

```
Stream<Integer> s = Stream.generate(() -> 42);  
// s = (42, 42, 42, 42, 42, 42, ...)
```

- Mittels Iterator: `Stream.iterate(T start, UnaryOperator<T> fun)`
 - Stream startet mit dem Wert `start` .
 - Jeder folgende Wert wird aus dem letzten Wert mithilfe von `fun` berechnet.

```
Stream<Integer> s = Stream.iterate(1, x -> x + 3);  
// s = (1, 4, 7, 10, 13, 16, 19, ...)
```

- Streams für primitive Datentypen: `IntStream`, `LongStream`, `DoubleStream`
 - Vermeidet Performanceverlust durch Autoboxing

```
IntStream s = IntStream.range(1, 8);           // s = (1, 2, 3, 4, 5, 6, 7)
IntStream s = IntStream.rangeClosed(1, 8);     // s = (1, 2, 3, 4, 5, 6, 7, 8)
```

- Aus den Zeilen einer Datei:

```
import java.nio.file.*;

Path path = Paths.get("osterhasen.txt");
try {
    Stream<String> zeilen = Files.lines(path);
    /* ... */
} catch (IOException e) { ... }
```

Stream ausgeben

- Ausgabe eines Streams mit `forEach` :

```
Stream<Integer> s = Stream.of(4, 8, 15, 16);

s.forEach(System.out::println); // jedes Element ausgeben
// Ausgabe: 4, 8, 15, 16

s.forEach(System.out::println); // nochmal jedes Element ausgeben?
// IllegalStateException: "stream has already been operated upon or closed"
```

Terminale Operation

```
Stream<Integer> s = Stream.of(4, 8, 15, 16);  
s.forEach(System.out::println); // jedes Element ausgeben  
s.forEach(System.out::println); // nochmal jedes Element ausgeben?  
// IllegalStateException: "stream has already been operated upon or closed"
```

- `forEach` ist eine **terminale Operation**.
- Eine terminale Operation verarbeitet den Stream und macht ihn dadurch **ungültig**.
- Streams können daher grundsätzlich nur ein Mal verarbeitet werden.

Größe eines Streams

- Erzeugung mittels Iterator: `Stream.iterate(T start, UnaryOperator<T> fun)`
 - Stream startet mit dem Wert `start`.
 - Jeder folgende Wert wird aus dem letzten Wert mithilfe von `fun` berechnet.

```
Stream<Integer> s = Stream.iterate(1, x -> x + 3);  
// s = (1, 4, 7, 10, 13, 16, 19, ...)
```

- Wie viele Elemente hat der resultierende Stream?

Größe eines Streams

- Wie viele Elemente hat der resultierende Stream?
- Versuch einer Ausgabe:

```
Stream<Integer> s = Stream.iterate(1, x -> x + 3);  
s.forEach(System.out::println)
```

- **Beobachtung:** Ausgabe terminiert nicht!
- Streams können potenziell unendlich lang sein.
- Beschränkung eines Streams auf endlich viele Elemente:

```
Stream<Integer> s = Stream.iterate(1, x -> x + 3).limit(10);  
s.forEach(System.out::println)  
// Ausgabe: 1, 4, 7, 10, 13, 16, 19, 22, 25, 28
```

Verarbeitung unendlicher Streams

```
Stream<Integer> s = Stream.iterate(1, x -> x + 3);  
s.forEach(System.out::println)
```

- **Frage:** Wieso terminiert die `iterate`-Methode des Streams in Zeile 1, obwohl der Stream danach unendlich viele Elemente hat?
- **Antwort:** `iterate` definiert einen Stream, verarbeitet ihn aber nicht.
- Die Verarbeitung des Streams beginnt erst, wenn eine terminale Operation ausgeführt wird.
- `iterate` ist eine **Zwischenoperation** (*intermediate operation*).

"Faule" Operationen

- Die Verarbeitung eines Streams benötigt drei Teile:
 - Eine Datenquelle (z.B. `Stream.of(...)`).
 - Beliebig viele Zwischenoperationen, die zunächst nicht ausgeführt werden.
 - Eine terminale Operation, die die Verarbeitung anstößt und den Typ des Ergebnisses bestimmt.
- Die Zwischenoperationen werden erst ausgeführt, wenn *tatsächlich* das Ergebnis der Verarbeitung benötigt wird.
- Dadurch ist es *überhaupt* erst möglich, mit unendlichen Streams zu arbeiten.
- Dieses Prinzip heißt **lazy evaluation** (*verzögerte Auswertung*).

Klassische Stream-Operationen: map

`<R> Stream<R> map(Function<? super T, ? extends R> mapper)` (*intermediate*)

- `map` wendet auf jedes Element des Streams eine Funktion (= Lambda-Ausdruck oder Methode) an.
- Signatur: Wenn `map` auf einen `Stream<T>` angewandt wird und ein `Stream<R>` entstehen soll, benötigt man eine Funktion, die einen Wert vom Typ `T` (oder Obertyp) konsumiert und einen Wert vom Typ `R` (oder Untertyp) zurückgibt.
- **Beispiel:** Bilde jeden String in einem Stream auf seine Länge ab.

```
Stream<String> stringStream = Stream.of("Hallo", "wie", "geht", "es", "dir", "?");  
Stream<Integer> laengen = stringStream.map(String::length);
```

Klassische Stream-Operationen: filter

```
Stream<T> filter(Predicate<? super T> predicate) (intermediate)
```

- `filter` tut, was es sagt: Behält Elemente, für die eine Bedingung `true` ist.
- Signatur: Wenn `filter` auf einen `Stream<T>` angewandt wird, entsteht ein neuer `Stream<T>`. Man benötigt ein **Prädikat** für `T`, d.h. eine Funktion, die einen Wert vom Typ `T` (oder Obertyp) konsumiert und `true` oder `false`.
- **Beispiel:** Behalte nur die durch 7 teilbaren `Integer` eines Streams.

```
IntStream is = IntStream.range(1,100).filter(x -> (x % 7 == 0))
```

Klassische Stream-Operationen: reduce

`T reduce(T identity, BinaryOperator<T> accumulator)` (*terminal*)

- `reduce` fasst einen `Stream<T>` zu einem einzigen Wert vom Typ `T` zusammen.
- Signatur: `identity` ist der Startwert für einen leeren Stream.
`accumulator` ist eine Funktion, die zwei Werte vom Typ `T` zu einem neuen Wert vom Typ `T` zusammenrechnet.
- **Beispiel:** Strings zusammenfügen.

```
Stream<String> s = Stream.of("Hallo", "wie", "geht", "es", "dir", "?");  
String zusammen = s.reduce("", (a,b) -> a + " " + b);
```

Weitere Stream-Operationen: peek

`Stream<T> peek(Consumer<? super T> action)` (*intermediate*)

- `peek` erstellt aus einem Stream einen neuen Stream und setzt dabei eine Kopie jedes Elements in eine Funktion ein.
- Signatur: `action` ist Funktion, die einen Wert vom Typ `T` konsumiert und nichts zurückgibt.
- **Beispiel:** Ergebnisse von Zwischenoperationen ausgeben.

```
Stream.iterate(1, x -> x+1).peek(System.out::println)
    .map(x -> 2*x).peek(System.out::println)
    .limit(5).reduce(0, (a, b) -> a+b);
```


Weitere Stream-Operationen: count

`long count()` (*terminal*)

- `count` liefert die Anzahl der Elemente des Streams.
- `s.count()` ist gleichbedeutend mit `s.map(x -> 1).reduce(0, (a, b) -> a+b)`

Weitere Stream-Operationen: dropWhile

```
Stream<T> dropWhile(Predicate<? super T> predicate) (intermediate)
```

- `dropWhile` entfernt die vordersten Elemente eines Streams, solange eine Bedingung erfüllt ist. Das erste Element, das die Bedingung **nicht** erfüllt, ist das erste Element des neuen Streams.
- **Beispiel:** Entferne alle vordersten Elemente bis zum ersten Element, das mindestens 1000 ist.

```
Stream.iterate(1, x -> x+3).limit(1000).dropWhile(x -> x < 1000)
```